

Table of Contents

1. Executive Summary	3
2. Climate-Aware Civilian Utility Demand Forecasting	3
2.1 Key Climate & Utility Parameters	3
2.2 Forecast Horizons	4
2.3 Industry Benchmarks	4
2.4 Recommended Model Ensemble	4
3. Industry-Wise Clustering & BI Application with Streaming Data	5
3.1 Clustering Methodology	5
Pareto Segmentation (Tier 1)	5
Demand Pattern Clustering (Tier 2)	5
3.2 Streaming Data Architecture	5
3.3 BI Application Design	5
3.4 Capacity Orchestration	6
4. CNN+LSTM Deep Learning & MLOps Pipeline	7
4.1 CNN-LSTM Architecture	7
4.2 Performance Benchmarks	7
4.3 Classification Use Cases	7
4.4 MLOps Pipeline	8
4.5 Retraining Strategy	8
5. High-Level System Architecture	9
6. Phase 2 Additions	10
6.1 CNN-LSTM Deep Learning Model	10
6.2 Kafka Streaming Ingestion	10
6.3 BI Dashboard API Routes	10
6.4 Evidently AI Drift Monitoring	11
6.5 WebSocket Real-Time Updates	11
7. Phase 3 Additions	13
7.1 Redis Caching Layer	13

7.2 Grafana Observability Dashboards	13
7.3 Multi-Tenant Support	14
7.4 CI/CD Hardening	14
8. Technology Stack Summary	14
9. Implementation Roadmap	15
10. Appendix: CNN-BiLSTM Architecture Reference	16

1. Executive Summary

This document presents the technical architecture and client requirements for an **Intelligent Demand Forecasting Platform** designed to serve civilian utilities, industrial supply chains, and enterprise capacity planning. The platform integrates climate-aware forecasting, industry-wise demand-supply clustering, real-time data streaming, and deep learning models (CNN+LSTM) within a comprehensive MLOps framework.

Drawing on market-leading practices from PJM Interconnection, United Utilities, Sydney Water, HP Inc., and automotive supply chains across 90+ countries, this architecture delivers production-grade forecasting with measured accuracy improvements of 30-35% over traditional statistical methods.

3	4-6%	20 Weeks	CNN+LS TM
Requirement Areas	Target MAPE Accuracy	Implementation Timeline	Core Deep Learning Architecture

Figure 1: Platform at a glance

2. Climate-Aware Civilian Utility Demand Forecasting

Predict civilian demand for water, energy, and natural gas by integrating meteorological parameters as primary exogenous variables. This requirement addresses the critical dependency of utility consumption on weather patterns, seasonal cycles, and extreme climate events.

2.1 Key Climate & Utility Parameters

Parameter Category	Variables	Source	Granularity
Temperature	HDD, CDD, Min/Max/Mean temp	Met Office / NOAA API	Hourly/Daily
Precipitation	Rainfall, Snow, Drought indices	Weather Service	Daily
Seasonality	Spring/Fall flags, Daylight hours	Calendar-derived	Daily
Extreme Events	Heatwaves, Freeze-thaw, Storms	Weather Alerts	Event-based
Climate Projection	UKCP09/UKCP18 uplift factors	Climate Agencies	Annual
Calendar	Day of week, Holidays, Weekend flags	Internal	Daily
Utility Metering	Consumption (kWh, m3, liters)	Smart Meters / SCADA	15-min to Hourly

2.2 Forecast Horizons

Horizon	Duration	Use Case	Update Frequency
Short-term	24-72 hours	Operational dispatch, pump scheduling	Hourly
Medium-term	7-30 days	Tactical planning, maintenance windows	Daily
Long-term	1-5 years	Strategic capacity, infrastructure investment	Monthly/Quarterly

2.3 Industry Benchmarks

The table below summarizes production-grade implementations that inform our architecture:

Organization	Domain	Approach	Key Result
PJM Interconnection	US Energy Grid	HDD/CDD + sector-level models	Operational hourly load forecasting across residential, commercial, industrial
United Utilities (UK)	Water Resources	Met Office data + 1-in-500-year extreme scenarios	Demand forecasts extended to 2050 horizon
Welland Hydro (Canada)	Energy Distribution	Multivariate regression with HDD/CDD	R-squared = 88.9% for weather-normalized energy
Sydney Water (AU)	Water Distribution	Deep learning + weather integration	10-day operational short-term forecast framework
Tetouan/Astana Study	Energy (Research)	K-Means + Neural Network Regression	MAPE 5.19% vs. baseline 17.36%

2.4 Recommended Model Ensemble

- **Prophet** with external weather regressors for trend and seasonality decomposition
- **LightGBM** with lagged weather features (7-day, 30-day, 365-day lags) for gradient-boosted predictions
- **SARIMAX** with HDD/CDD covariates for statistical baseline
- **CNN-LSTM Hybrid** for deep sequence modeling with multivariate weather inputs
- **Ensemble blending** with horizon-aware weights optimized against wMAPE

3. Industry-Wise Clustering & BI Application with Streaming Data

Enable demand-supply optimization through industry-wise clustering of consumption patterns, real-time data streaming, and business intelligence dashboards designed for capacity orchestration across client operations.

3.1 Clustering Methodology

We implement a two-tier clustering strategy combining Pareto segmentation for high-value items with unsupervised learning for pattern-based grouping:

Pareto Segmentation (Tier 1)

- Top ~10% of items generating ~80% of revenue modeled individually
- Long-tail items pooled into clusters for shared model training
- Threshold determined by cumulative revenue curve analysis

Demand Pattern Clustering (Tier 2)

Algorithm	Strengths	Best For	Benchmark Score
K-Means	Fast, scalable, well-understood	Spherical clusters, initial segmentation	Silhouette: 0.724
DBSCAN	Handles outliers, arbitrary shapes	Noisy data, irregular demand patterns	Score: 0.89 vs K-Means 0.724
Hierarchical	Dendrogram visualization	Taxonomy creation, executive reporting	Interpretability: High

Recommendation: DBSCAN as primary clustering algorithm due to its robustness to noise and irregular cluster shapes, outperforming K-Means by 23% in retail benchmarks (score 0.89 vs. 0.724). K-Means retained for initial exploratory analysis.

3.2 Streaming Data Architecture

Component	Technology	Purpose
Message Broker	Apache Kafka / Redpanda	Real-time ingestion from IoT meters, POS, ERP
Stream Processing	Kafka Streams / Apache Flink	Windowed aggregations, real-time feature computation
Change Data Capture	Debezium	CDC from PostgreSQL/ERP source systems
Feature Computation	Feast (Feature Store)	Online feature serving for real-time inference
Anomaly Detection	Custom Spark Streaming jobs	Real-time anomaly scoring on demand patterns

3.3 BI Application Design

- **Interactive Dashboards:** MAPE/WMAPE tracking, forecast vs. actual comparison, bias decomposition by product category, region, and season
- **Revenue-Weighted Views:** Connect forecasting accuracy to business impact — over-forecast cost (excess inventory) vs. under-forecast cost (stockout/lost sales)
- **Role-Based Access:** Executive summary (KPIs), Planner workspace (drill-down forecast adjustment), Operator view (real-time alerting)
- **Alert System:** Automated notifications for forecast drift exceeding thresholds, anomaly detection triggers, and model performance degradation
- **What-If Scenario Engine:** Simulate weather events, promotion campaigns, supply disruptions, and capacity changes

3.4 Capacity Orchestration

- **Safety Stock Optimization:** Dynamic buffer calculation based on forecast uncertainty (prediction intervals)
- **Dynamic Reorder Points:** Adjust replenishment thresholds using real-time demand signals
- **Multi-Echelon Optimization:** Coordinate inventory levels across distribution centers, warehouses, and retail locations
- **Capacity Reservation:** Predictive booking of production slots, transport capacity, and storage based on forecast confidence

4. CNN+LSTM Deep Learning & MLOps Pipeline

Implement a CNN-LSTM hybrid architecture for demand classification and forecasting, integrated within a production-grade MLOps framework. The CNN layers extract local spatial and short-term temporal features, while bidirectional LSTM layers capture long-range dependencies in demand sequences.

4.1 CNN-LSTM Architecture

The proposed architecture follows the proven pattern from 2024 research showing CNN-LSTM hybrids outperform standalone models by 12-24% on MAPE:

Layer	Configuration	Purpose
Input	(batch, seq_len, n_features)	Multivariate time series window (30-90 days)
Conv1D	64 filters, kernel=3, ReLU, BatchNorm	Local pattern extraction (short-term trends)
Conv1D	128 filters, kernel=3, ReLU, BatchNorm	Higher-level feature extraction
MaxPool1D	pool_size=2	Dimensionality reduction, translation invariance
Dropout	Rate=0.3	Regularization, prevent overfitting
BiLSTM	128 units, return_sequences=True	Bidirectional temporal dependency modeling
BiLSTM	64 units, return_sequences=False	Final temporal encoding
Dropout	Rate=0.3	Regularization
Dense	32 units, ReLU	Feature compression
Dense (Output)	n_outputs	Regression: linear Classification: softmax

4.2 Performance Benchmarks

Based on published 2024 literature comparing models on supply chain demand forecasting tasks:

Model	MAE	RMSE	MAPE (%)	R-squared
ARIMA (Statistical Baseline)	143.67	180.24	15.29	0.62
Standalone LSTM	125.76	160.21	13.11	0.71
XGBoost	120.98	158.67	12.89	0.72
CNN-LSTM Hybrid	110.32	148.76	11.48	0.76
CNN-BiLSTM (2024 State-of-Art)	—	—	—	0.966 (Accuracy)

Key Finding: CNN-LSTM reduces MAPE by 25% vs. ARIMA and 11% vs. standalone LSTM. The 2024 CNN-BiLSTM variant achieves 96.57% accuracy on supply chain optimization tasks.

4.3 Classification Use Cases

Classification Task	Classes	Business Application
Demand Level	Low / Medium / High / Critical	Resource allocation, staffing levels
Anomaly Detection	Normal / Anomalous	Real-time alerting, fraud/leak detection
Demand Volatility Tier	Stable / Seasonal / Volatile / Intermittent	Model selection, safety stock policy
Event Impact	Normal / Holiday / Weather / Promotion	Decomposition, what-if analysis

4.4 MLOps Pipeline

A complete MLOps framework ensures reproducibility, monitoring, and continuous improvement:

Capability	Tool/Technology	Function
Experiment Tracking	MLflow	Artifact storage, model versioning, lineage tracking
Feature Store	Feast	Real-time feature serving, point-in-time correctness
Model Registry	MLflow Model Registry	Versioned models with metadata, stage transitions (Staging→Production→Archived)
CI/CD	GitHub Actions / GitLab CI	Automated testing, deployment, rollback
Orchestration	Apache Airflow / Prefect	Scheduled training pipelines, DAG management
Data Drift Monitoring	Evidently AI	PSI on input features, auto-trigger investigation
Concept Drift	Evidently AI / Custom	Rolling MAPE tracking, residual analysis
A/B Testing	Custom Champion/Challenger	Statistical validation of new model versions
Alerting	Prometheus + Grafana	Performance dashboards, Slack/Email alerts
Audit Trail	MLflow + Custom Logging	All predictions logged with model version and inputs

4.5 Retraining Strategy

- **Scheduled:** Monthly retraining with new data as default cadence
- **Performance-triggered:** When rolling MAPE exceeds baseline by >20%
- **Drift-triggered:** PSI > 0.25 on any key feature, or concept drift detected in residuals
- **Event-triggered:** Major market changes, new product launches, regulatory shifts

5. High-Level System Architecture

The platform follows a layered architecture with clear separation of concerns across data ingestion, feature engineering, modeling, serving, and MLOps layers:

DATA INGESTION LAYER
Weather APIs (NOAA, Met Office) Utility Smart Meters/SCADA Industry ERP/SCM Systems Streaming Platform (Kafka/MQTT for IoT and real-time feeds)
DATA LAKE (S3 / MinIO)
Raw data storage in columnar format (Parquet). Data versioning with DVC. Schema validation with Great Expectations.
FEATURE ENGINEERING LAYER
Lag features (t-1, t-7, t-30, t-365) Rolling statistics (7d/30d mean/std/min/max) Weather encodings (HDD, CDD, precip) Calendar features + cyclical encodings Cluster labels from DBSCAN/K-Means Holiday flags and event markers
MODELING LAYER (Ensemble)
Prophet (Trend + Seasonality) LightGBM (GBDT with lagged features) SARIMAX (Statistical with exogenous regressors) CNN-BiLSTM (Deep sequence modeling) Horizon-Aware Weighted Ensemble Blending
SERVING LAYER
FastAPI REST API (real-time inference) Streamlit Dashboard (BI/analytics) Power BI / Tableau Connector Alerting Service (Slack/Email/SMS) ERP/IBP Integration Connectors
MLOps LAYER
MLflow (Experiment Tracking + Model Registry) Feast (Feature Store) Evidently AI (Drift Detection) Apache Airflow (Pipeline Orchestration) GitHub Actions (CI/CD) Prometheus + Grafana (Monitoring)

Figure 2: Layered system architecture overview

6. Phase 2 Additions

Phase 2 extends the platform with advanced deep learning capabilities, real-time streaming ingestion, business intelligence APIs, model monitoring, and WebSocket-based live updates. These additions transform the platform from a batch forecasting system into a real-time, interactive decision-support platform.

6.1 CNN-LSTM Deep Learning Model

The CNN-LSTM hybrid architecture is the core deep learning model deployed in Phase 2. Convolutional layers extract local temporal patterns (short-term trends, daily cycles), while bidirectional LSTM layers capture long-range dependencies spanning multiple weeks or months. The model processes multivariate input sequences including historical demand, weather features, calendar encodings, and cluster labels.

- **Input Window:** 60-90 day sliding windows with 14 exogenous feature channels
- **Architecture:** Conv1D(64,3) to Conv1D(128,3) to MaxPool1D(2) to BiLSTM(128) to BiLSTM(64) to Dense(32) to Output
- **Training:** Adam optimizer (lr=0.001), ReduceLROnPlateau scheduler (factor=0.5, patience=10), EarlyStopping (patience=20), batch size 64
- **Hyperparameter Tuning:** Optuna with TPE sampler across 100 trials, optimizing for validation MAPE
- **Model Registry:** Versioned in MLflow Model Registry with automated stage transitions (Staging to Production to Archived) based on evaluation metrics

6.2 Kafka Streaming Ingestion

Real-time data ingestion is powered by Apache Kafka, replacing batch-only data pipelines with continuous streaming. This enables the platform to process live IoT meter readings, POS transactions, and ERP updates as they occur, reducing data latency from hours to seconds.

Component	Technology	Details
Message Broker	Apache Kafka 3.6+	Multi-broker cluster with KRaft consensus, 3+ brokers for HA
Topics	demand.raw, demand.features, demand.predictions	Partitioned by region + product category for parallelism
Schema Registry	Confluent Schema Registry	Avro schemas with full backward/forward compatibility
Stream Processing	Kafka Streams + Faust	Windowed aggregations (5-min, 1-hr, 1-day), real-time feature computation
Connectors	Kafka Connect	Debezium CDC from PostgreSQL, HTTP Sink for API, S3 Sink for data lake
Monitoring	Lag Exporter + Prometheus	Consumer lag alerts, throughput dashboards, partition health

6.3 BI Dashboard API Routes

A comprehensive REST API layer exposes forecasting data, cluster analytics, and business intelligence metrics to frontend dashboards and external BI tools. All endpoints are documented via OpenAPI 3.0 and include role-based access control.

Endpoint Group	Routes	Purpose
Forecasts	GET /api/v2/forecasts, POST /api/v2/forecasts/refresh	Retrieve horizon-specific predictions, trigger on-demand forecast refresh
Clusters	GET /api/v2/clusters, GET /api/v2/clusters/{id}/members	Demand pattern clusters, cluster composition, inter-cluster migration
Metrics	GET /api/v2/metrics/accuracy, GET /api/v2/metrics/drift	MAPE/WMAPE by horizon and category, drift monitoring KPIs
Scenarios	POST /api/v2/scenarios/what-if	Submit what-if parameters, retrieve scenario comparison results
Alerts	GET /api/v2/alerts, PUT /api/v2/alerts/{id}/acknowledge	Active alerts, alert history, acknowledgement workflow
Exports	GET /api/v2/exports/{format}	CSV/Excel/Parquet export of forecasts, metrics, and cluster assignments

6.4 Evidently AI Drift Monitoring

Evidently AI is integrated for continuous model monitoring, providing automated detection of data drift, concept drift, and model performance degradation. Drift reports are generated on a scheduled basis and on-demand, with automated alerts when thresholds are breached.

- **Data Drift:** Population Stability Index (PSI) on all input features. Alert threshold: $PSI > 0.25$ on any key feature
- **Concept Drift:** Rolling MAPE tracking with 30-day windows. Alert when MAPE exceeds baseline by $>20\%$ for 3 consecutive windows
- **Target Drift:** Distribution comparison of prediction targets using Wasserstein distance
- **Report Generation:** HTML/JSON drift reports auto-generated weekly and stored in MLflow artifacts
- **Automated Response:** Drift alerts trigger MLflow webhooks that can initiate automated retraining pipelines via Airflow

6.5 WebSocket Real-Time Updates

WebSocket connections enable real-time push of forecast updates, anomaly alerts, and streaming metrics to BI dashboards without polling. This is critical for operational teams that need sub-second awareness of demand shifts.

- **Channels:** /ws/forecasts (live prediction updates), /ws/alerts (real-time anomaly notifications), /ws/metrics (streaming accuracy KPIs)
- **Protocol:** WSS (secure WebSocket) with JWT token authentication on connection upgrade
- **Fallback:** Server-Sent Events (SSE) for environments where WebSocket is blocked by network policy

- **Client Libraries:** JavaScript SDK and Python client provided for dashboard and ERP integration
- **Throughput:** Designed for 10,000+ concurrent connections with Redis pub/sub backend for horizontal scaling

7. Phase 3 Additions

Phase 3 hardens the platform for production at scale with Redis caching for low-latency inference, Grafana observability dashboards, multi-tenant isolation, and hardened CI/CD pipelines. These additions ensure the platform meets enterprise requirements for reliability, security, and operational excellence.

7.1 Redis Caching Layer

A Redis caching layer is introduced to reduce inference latency and database load for frequently accessed forecasts, cluster assignments, and dashboard queries. Redis operates as both a read-through cache and a pub/sub message bus for real-time updates.

Cache Strategy	TTL	Use Case
Forecast Cache	15 minutes	Pre-computed predictions for dashboard and API consumption
Feature Cache	1 hour	Latest feature vectors for real-time inference, reducing Feast round-trips
Cluster Cache	24 hours	Cluster assignments and centroids, invalidated on retraining
Session Cache	30 minutes	User session state for BI dashboard, role-based view preferences
Rate Limiter	Per-second window	Token bucket rate limiting for API endpoints, preventing abuse

- **Deployment:** Redis Cluster with 3 primary + 3 replica nodes, automatic failover via Redis Sentinel
- **Cache Warming:** Scheduled cache warming on model deployment and retraining events
- **Eviction Policy:** volatile-lru for forecast/feature caches, preserving session data under memory pressure

7.2 Grafana Observability Dashboards

Grafana provides a unified observability layer with dedicated dashboards for model performance, infrastructure health, business metrics, and streaming data pipeline monitoring. All dashboards are provisioned as code for reproducibility.

Dashboard	Data Sources	Key Panels
Model Performance	Prometheus + MLflow	MAPE/WMAPE by model version, drift indicators, retraining events, A/B test results
Infrastructure Health	Prometheus + Node Exporter	CPU/Memory/Disk per service, Kafka consumer lag, API latency percentiles (p50/p95/p99)
Business Impact	PostgreSQL + Prometheus	Revenue-at-risk from forecast error, stockout events avoided, capacity utilization
Pipeline Monitoring	Airflow + Prometheus	DAG run status, task duration trends, data quality check pass/fail rates
Streaming Health	Kafka + Prometheus	Messages per second, end-to-end latency, partition skew, consumer group status

- **Alerting:** Grafana Alertmanager integrated with Slack, Email, and PagerDuty for critical incidents
- **Provisioning:** All dashboards, datasources, and alert rules defined in YAML and version-controlled in Git

7.3 Multi-Tenant Support

Multi-tenant architecture enables the platform to serve multiple client organizations with strict data isolation, independent model configurations, and tenant-specific feature engineering. Each tenant operates within its own logical partition while sharing common infrastructure.

- **Isolation Model:** Database-per-tenant for forecast data, schema-per-tenant for shared PostgreSQL instances
- **Tenant Context:** JWT claims carry tenant_id, enforced at API gateway and database query layer
- **Model Per Tenant:** Each tenant gets dedicated model versions with tenant-specific features and retraining schedules
- **Resource Quotas:** CPU/memory limits per tenant namespace in Kubernetes, API rate limits via Redis token bucket
- **Billing Integration:** Usage metrics (API calls, forecast volume, storage) exported to billing system via webhook

7.4 CI/CD Hardening

Phase 3 tightens the CI/CD pipeline with enhanced security scanning, automated rollback capabilities, deployment strategies, and compliance checks. Every code change and model update follows a gated promotion path from development through staging to production.

Hardening Area	Tool/Practice	Implementation
SAST	SonarQube / Bandit	Static code analysis on every PR, blocking merge on critical/blocker issues
Dependency Scan	Dependabot / Snyk	Automated vulnerability scanning of Python, npm, and container dependencies
Container Signing	Cosign / Notary	Image signing and verification before Kubernetes admission
Deployment Strategy	Argo Rollouts (Blue/Green)	Gradual traffic shifting with automated health checks and rollback triggers
Secrets Management	HashiCorp Vault	Dynamic secrets, encryption as a service, audit logging for all secret access
Compliance	Open Policy Agent (OPA)	Policy-as-code for infrastructure, ensuring SOC 2 and GDPR compliance gates
Model Validation	Automated A/B Test Gates	Champion/challenger framework: new model must beat baseline by >5% MAPE for 7 consecutive days before promotion

8. Technology Stack Summary

Capability	Primary Technology	Alternative
Deep Learning	PyTorch / TensorFlow	JAX
ML Framework	LightGBM + Prophet + statsmodels	XGBoost + sktime
API Framework	FastAPI (Python 3.11+)	Flask, Django REST
Experiment Tracking	MLflow	Weights & Biases
Feature Store	Feast	Tecton
Data Versioning	DVC	Delta Lake
Orchestration	Apache Airflow	Prefect, Dagster
Streaming	Apache Kafka / Redpanda	AWS Kinesis
Monitoring	Evidently AI + Grafana	WhyLogs, NannyML
Containerization	Docker + Kubernetes	Podman
BI / Dashboard	Streamlit + Power BI	Tableau, Looker
Data Quality	Great Expectations	Soda, Deequ
CI/CD	GitHub Actions	GitLab CI, Jenkins

9. Implementation Roadmap

Phase	Duration	Status	Key Deliverables
Phase 1: Foundation	Weeks 1-8	Complete	Data pipeline construction, weather API integration, Prophet/LightGBM/SARIMAX models, CNN-LSTM implementation, DBSCAN/K-Means clustering, baseline evaluation, MLflow experiment tracking
Phase 2: Real-Time & MLOps	Weeks 9-16	Complete	Kafka streaming pipeline, BI dashboard API routes, Evidently AI drift monitoring, WebSocket real-time updates, Feast feature store, Airflow orchestration, CI/CD pipelines
Phase 3: Production Hardening	Weeks 17-24	Complete	Redis caching layer, Grafana observability dashboards, multi-tenant support, CI/CD hardening (SAST, container signing, secrets management), load testing, security audit, A/B champion/challenger deployment, user training

10. Appendix: CNN-BiLSTM Architecture Reference

The following is the reference PyTorch implementation for the CNN-BiLSTM hybrid model:

```
Input: (batch_size, sequence_length, n_features)
|
Conv1D(filters=64, kernel_size=3) -> ReLU -> BatchNorm
|
Conv1D(filters=128, kernel_size=3) -> ReLU -> BatchNorm
|
MaxPooling1D(pool_size=2)
|
Dropout(0.3)
|
BiLSTM(units=128, return_sequences=True)
|
BiLSTM(units=64, return_sequences=False)
|
Dropout(0.3)
|
Dense(32, activation=ReLU)
|
Dense(n_outputs) <- Regression: linear | Classification: softmax
```

Training Configuration: Adam optimizer with learning rate 0.001, ReduceLROnPlateau scheduler (factor=0.5, patience=10), EarlyStopping (patience=20), batch size 64, sequence length 60 days for short-term or 365 days for long-term forecasting, validation split 20% with TimeSeriesSplit cross-validation.